

Divide and Conquer and Solving Recurrences

Design and Analysis of Algorithms

Brian C. Dean

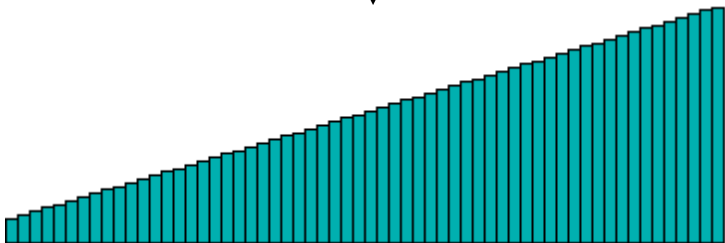
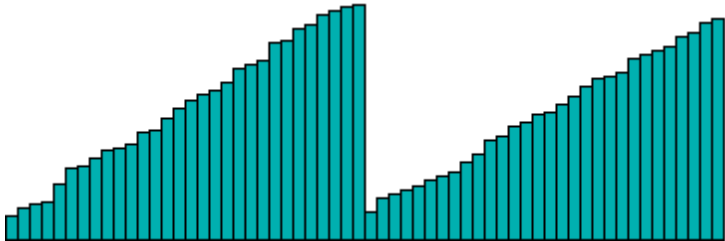
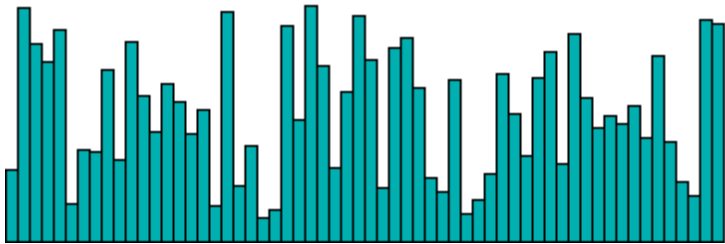
**School of Computing
Clemson University**



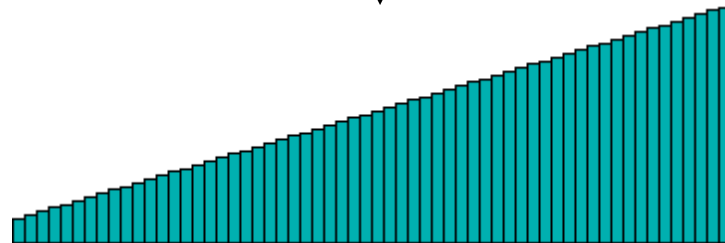
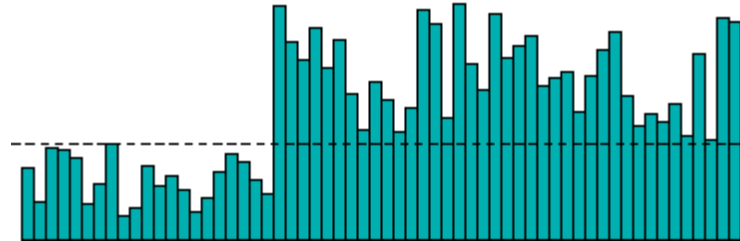
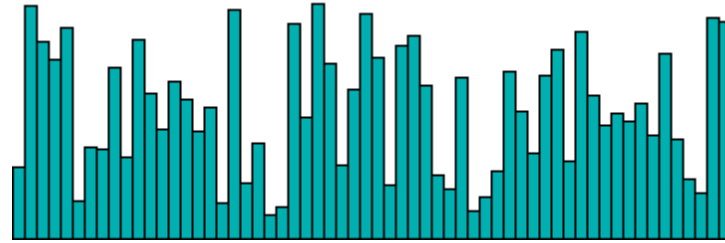
Divide and Conquer

- Extremely powerful and widely-applicable technique for designing algorithms:
 - **Divide** problem into “multiplicatively” smaller subproblems (e.g., 2 problems of size $n/2$) that have the same structure as the original.
 - **Recursively solve** the subproblems.
 - **Combine** their solutions to obtain a solution to the original problem.
- In this discussion: how to analyze running times of such recursive algorithms using **recurrences**.

Divide and Conquer Examples



Merge Sort



QuickSort

Recurrences

- To merge sort an array of size n ...
 - We recursively sort two smaller arrays of size $n/2$.
 - Then we spend $\Theta(n)$ time merging the results.
- If $T(n)$ denotes the running time of merge sort on an input of size n , we can therefore write a **recurrence** (recursive formula) for $T(n)$:

$$T(n) = 2T(n/2) + \Theta(n)$$

As a base case, $T(n) = O(1)$ for $n = O(1)$.

- The goal is now to **solve** the recurrence to produce an explicit (non-recursive) formula for $T(n)$:

$$T(n) = \Theta(n \log n).$$

Simplifying Recurrences

- Actual merge sort recurrence:

$$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n)$$

- However, since we only care about an asymptotic solution, we can always ignore floors, ceilings, and any other small additive terms (e.g., $T(n/2 + 7)$ in our recursive calls).
- We can also replace the $\Theta(n)$ with just n , as this will only change our solution by a constant factor.
- So we focus on solving $T(n) = 2T(n/2) + n$.

Algebraic Expansion

- A useful way to solve (or guess the solution to) any recurrence is to simply expand it out a few levels:

$$\begin{aligned}T(n) &= n + 2T(n/2) \\&= n + 2[n/2 + 2T(n/4)] \\&= n + n + 4T(n/4) \\&= n + n + 4[n/4 + 2T(n/8)] \\&= n + n + n + 8T(n/8) \\&\dots \\&= \underbrace{n + n + n + \dots + n}_{n \log n} + \underbrace{nT(1)}_{n\Theta(1)} = \Theta(n \log n).\end{aligned}$$

Solving Simple Recurrences

- Let's start by considering “divide and conquer” recurrences of the form:

$$T(n) = aT(n/b) + f(n)$$

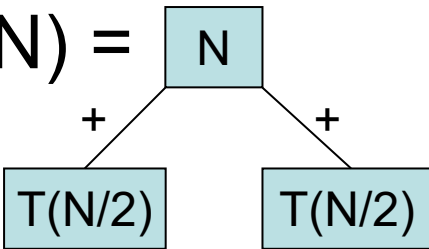
with $T(n) = O(1)$ for $n = O(1)$ as a base case.

- I.e., to solve a problem of size n , we recursively solve a subproblems of size n/b , and spend $f(n)$ time during the “division” and “recombination” steps of the algorithm.
- The simple form above covers nearly all of the recurrences we'll encounter in this course...

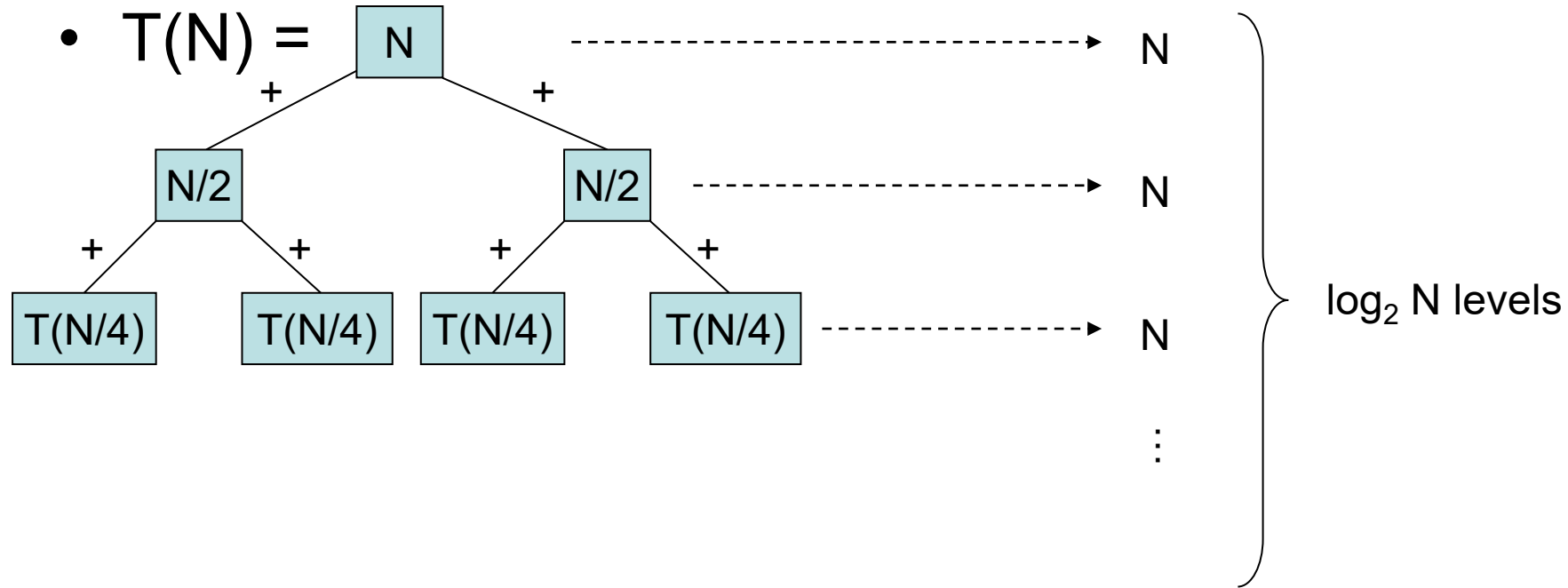
Tree Expansions

- $T(N) = N + 2T(N/2)$

Tree Expansions

- $T(N) =$ 

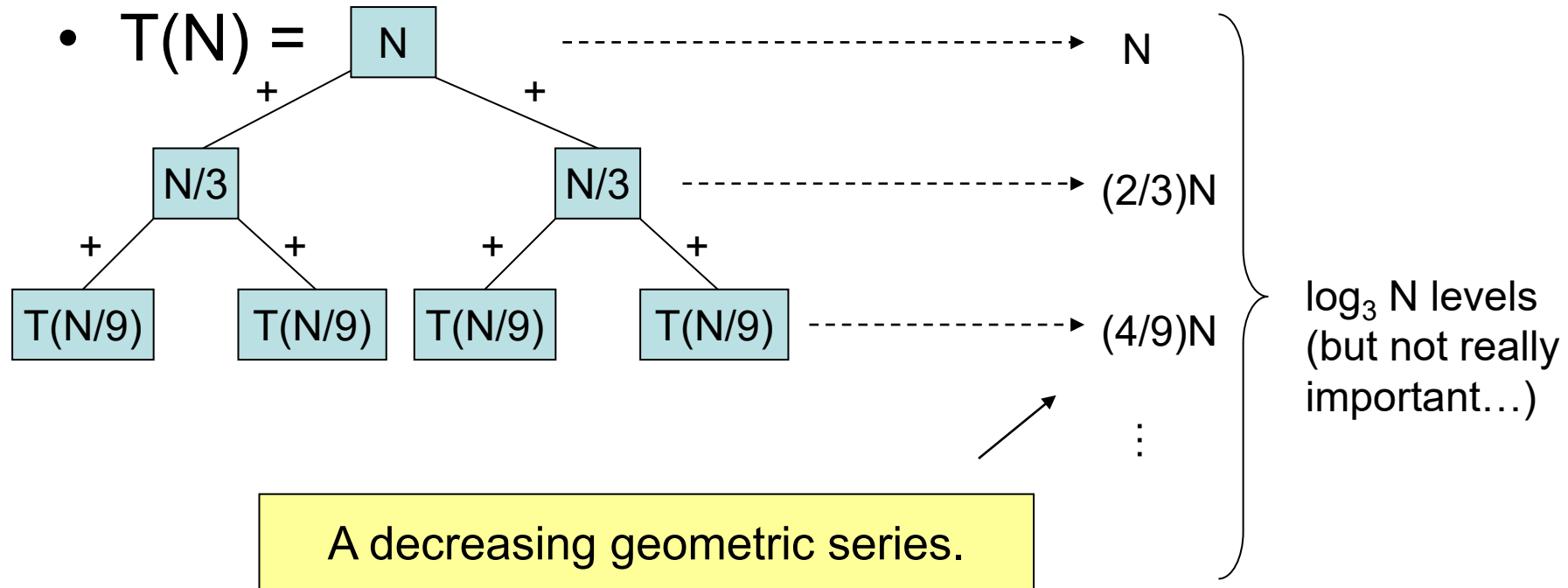
Tree Expansions



What about...

- $T(N) = N + 2T(N/3)$

Tree Expansions



Quick Aside: Asymptotic Behavior of a Geometric Series

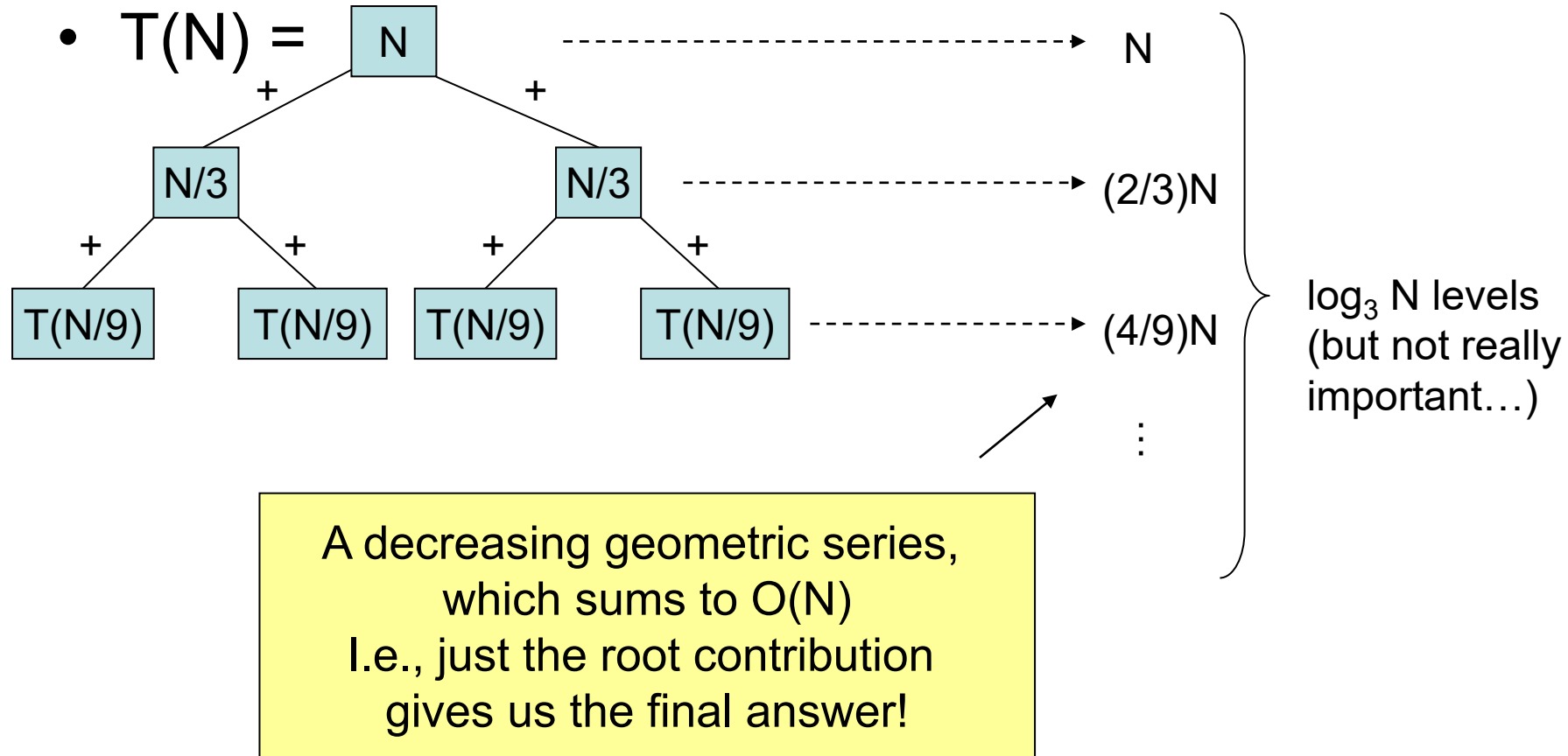
- Anyone remember how to sum a geometric series?

$$\begin{aligned} S &= n^2 + (3/4) n^2 + (3/4)^2 n^2 + \dots + (3/4)^k n^2 \\ &= n^2 [1 + (3/4) + (3/4)^2 + \dots + (3/4)^k] \\ &\leq n^2 [1 + (3/4) + (3/4)^2 + \dots] \\ &= 4n^2. \end{aligned}$$

- A decreasing geometric series behaves asymptotically just like its 1st term: $S = \Theta(n^2)$.
- By symmetry, if S were increasing, it would behave asymptotically like its final term:

$$T = n^2 + 2n^2 + \dots + 2^k n^2 = \Theta(2^k n^2).$$

Tree Expansions



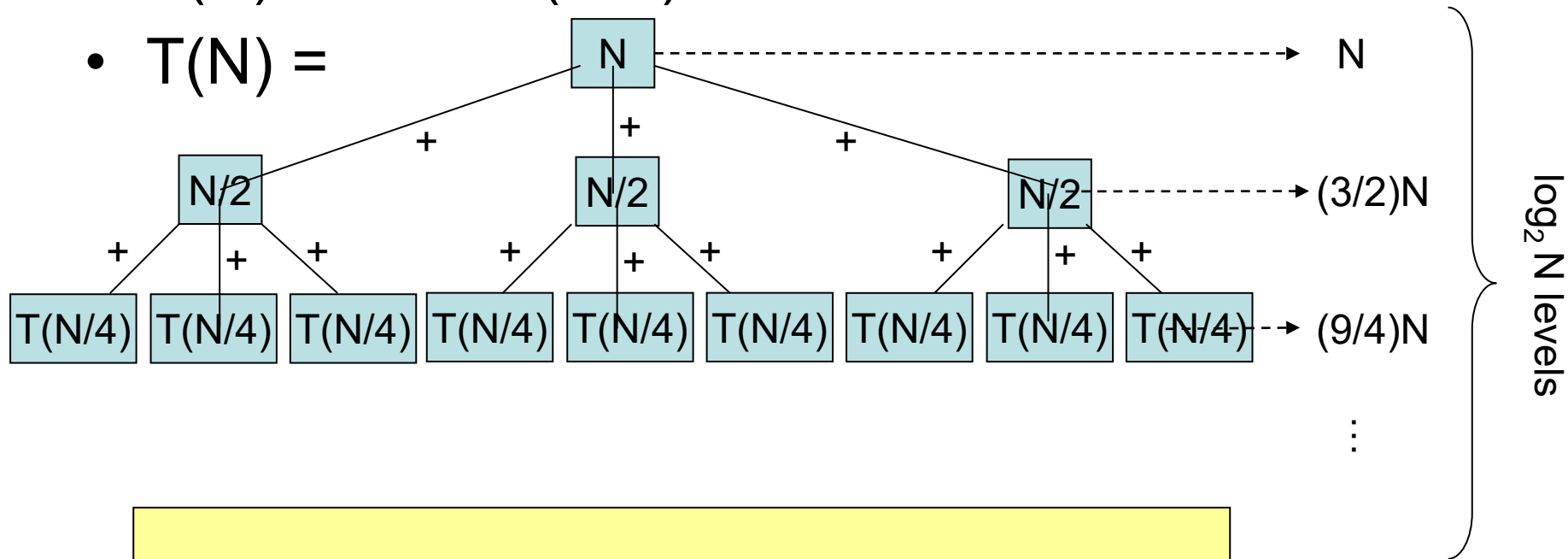
What about...

- $T(N) = N + 3T(N/2)$

The Increasing Case...

- $T(N) = N + 3T(N/2)$

- $T(N) =$



Increasing geometric series; sum is
 $\Theta(\text{contribution of leaves}) = \Theta(n^{\log_2 3})$

Tree Expansions

- Consider $T(n) = aT(n/b) + n^\alpha$
- If we add up the contribution of each level in the tree, we always obtain a geometric series!
- So we just need to know if this series is:
 - Decreasing: solution $T(n) = \Theta(\text{root contribution}) = \Theta(n^\alpha)$
 - Increasing: solution $T(n) = \Theta(\text{leaf contribution}) = \Theta(n^{\log_b a})$
 - Unchanging: solution $T(n) = \Theta(n^\alpha \log n)$, since $\Theta(n^\alpha)$ is the contribution on each of the $\log n$ levels.

The “Master Method”

- Consider $T(n) = aT(n/b) + n^\alpha$.
- There are $\log_b n$ levels in the tree, and our branching factor is a , so the total # of leaves is $a^{\log_b n} = n^{\log_b a}$.
- Since each leaf contributes $\Theta(1)$, the total leaf contribution is $\Theta(n^p)$, where $p = \log_b a$.
- We can now say that $T(n) =$

$\Theta(n^\alpha)$	if $\alpha > p$	(decreasing series)
$\Theta(n^\alpha \log n)$	if $\alpha = p$	(unchanging series)
$\Theta(n^p)$	if $\alpha < p$	(increasing series)
- This formula is sometimes called the **master method** for solving a divide-and conquer recurrence.

Practice...

Let's solve some recurrences...